

Dynamic Programming

Knapsack

Input: List of n items with weights w_i & value v_i and a knapsack with capacity W

Task: Pack items in the knapsack maximizing the total value

$W = 10$	Item	Weight	Value
	1	6	\$30
	2	3	\$14
	3	4	\$16
	4	2	\$9

Only 1 quantity of each item available:

One item 1 and One item 3 – 46 Dollars

Unlimited quantity of each item available:

One item 1 and Two item 4 – 48 Dollars

Note: Greedy (value) won't work – Example: largest value

$W=20$, Item 1 ($w=11$, $v=15$), Item 2 ($w=10$, $v=10$), Item 3 ($w=10$, $v=10$)

subproblems?

smaller knapsack capacities

fewer items

Knapsack with repetition

$K(w)$ = maximum value achievable with a knapsack of capacity w

optimal solution to $K(w)$ ← item i

removing this item



optimal solution to $K(w - w_i)$

→ $K(w) \leftarrow K(w - w_i) + v_i$

Which i ?
$$K(w) = \max_{i:w_i \leq w} \{K(w - w_i) + v_i\}$$

$$K(w) = \max\{\text{knapsack}(w - w_i) + v_i : w_i \leq w\}$$

Algorithm:

```
 $K(0) = 0$   
for  $w = 1$  to  $W$ :  
     $K(w) = \max\{K(w - w_i) + v_i : w_i \leq w\}$   
return  $K(W)$ 
```

Include 0 as well
with the max!

Bookkeeping

$O(n)$

Time Complexity: $O(nW)$

This algorithm fills in a one-dimensional table of length $W + 1$, in left-to-right order.

Memory: $O(W)$

Note: The input is usually considered to be n items and $\log_2(W)$ bits representing the weight. Therefore, the time complexity is actually exponential with respect to the input size (Pseudolinear!).

Knapsack in this form is just a variant of finding the longest path in DAG!

Knapsack without repetition

a second parameter, $0 \leq j \leq n$

$K(w, j)$ = maximum value achievable using a knapsack of capacity w and items $1, \dots, j$.

Solution: $K(W, n)$

$$K(w, j) = \max\{K(w - w_j, j - 1) + v_j, K(w, j - 1)\}$$

only if $w_j \leq w$

$\swarrow$
 $\downarrow$
 j is taken

$\downarrow$
 j is not taken

subproblems $K(\cdot, j - 1)$

2D table: $n + 1 \times W + 1$

Initialize all $K(0, j) = 0$ and all $K(w, 0) = 0$

for $j = 1$ to n :

for $w = 1$ to W :

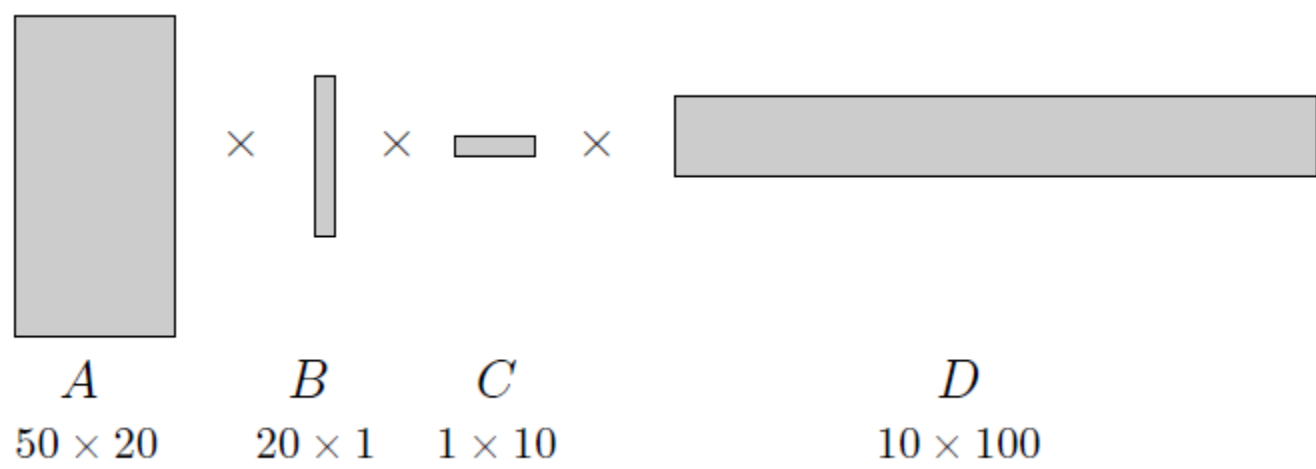
if $w_j > w$: $K(w, j) = K(w, j - 1)$

else: $K(w, j) = \max\{K(w, j - 1), K(w - w_j, j - 1) + v_j\}$

return $K(W, n)$

Time: $O(nW)$ Memory: $O(W)$ [bu], $O(nW)$ [td]

Chain Matrix Multiplication



$$A \times B \times C \times D$$

associative

Parenthesization	Cost computation	Cost
$A \times ((B \times C) \times D)$	$20 \cdot 1 \cdot 10 + 20 \cdot 10 \cdot 100 + 50 \cdot 20 \cdot 100$	120,200
$(A \times (B \times C)) \times D$	$20 \cdot 1 \cdot 10 + 50 \cdot 20 \cdot 10 + 50 \cdot 10 \cdot 100$	60,200
$(A \times B) \times (C \times D)$	$50 \cdot 20 \cdot 1 + 1 \cdot 10 \cdot 100 + 50 \cdot 1 \cdot 100$	7,000

The Greedy approach of smallest cost first does not work!

Consider:

$$A_1 \times A_2 \times \cdots \times A_n$$

dimensions

$$m_0 \times m_1, m_1 \times m_2, \dots, m_{n-1} \times m_n$$

subproblems $\longrightarrow A_i \times A_{i+1} \times \cdots \times A_j \quad 1 \leq i \leq j \leq n$

$C(i, j)$ = minimum cost of multiplying $A_i \times A_{i+1} \times \cdots \times A_j$

Number of multiplications involved: $|j - i|$

smallest $\longrightarrow i = j$

$$C(i, i) = 0$$

$j > i \longrightarrow C(i, j)$

Sol: $C(1, n)$

$$A_i \times A_{i+1} \times \cdots \times A_j$$

$$A_i \times \cdots \times A_k \quad A_{k+1} \times \cdots \times A_j$$

$$i \leq k \leq j$$

$O(n^2)$ storing

$$C(i, j) = C(i, k) + C(k + 1, j) + m_{i-1} \cdot m_k \cdot m_j$$

Optimal: $C(i, j) = \min_{i \leq k < j} \{C(i, k) + C(k + 1, j) + m_{i-1} \cdot m_k \cdot m_j\}$

for $i = 1$ to n : $C(i, i) = 0$

for $s = 1$ to $n - 1$:

Running complexity: $O(n^3)$

for $i = 1$ to $n - s$:

$$j = i + s$$

$$C(i, j) = \min\{C(i, k) + C(k + 1, j) + m_{i-1} \cdot m_k \cdot m_j : i \leq k < j\}$$

return $C(1, n)$

Bookkeeping: store k (multiplication #)

Travelling Salesman Problem

There are n places $\{1, 2, \dots, n\}$ and the distance between each pair of places is given by d_{ij} (complete graph).

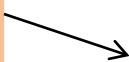
Goal: Starting from place 1, visit all the places exactly once by travelling the minimum distance in total.

$(n - 1)!$ possibilities

So an exhaustive algo will take: $O((n - 1)!)$

Lets see what dynamic programming can do... subproblems?

$S \subseteq \{1, 2, \dots, n\}$ with $1, j \in S$

$C(S, j)$ 

The minimum distance travelled to visit each place in S exactly once starting from 1 and ending at j .

$$|S| > 1 \downarrow$$

$$\text{If } |S| = 1, C(\{1\}, 1) = 0$$

$$C(S, 1) = \infty \longrightarrow \text{Cannot end at the starting place 1}$$

$$C(S, j) \longleftarrow C(S - \{j\}, i) \quad i \in S$$

$$C(S, j) = \min_{i \in S: i \neq j} C(S - \{j\}, i) + d_{ij} \longrightarrow \text{Travelling from } i \text{ to } j.$$

$$C(\{1\}, 1) = 0$$

for $s = 2$ to n :

for all subsets $S \subseteq \{1, 2, \dots, n\}$ of size s and containing 1:

$$C(S, 1) = \infty$$

for all $j \in S, j \neq 1$:

Bookkeeping

$$C(S, j) = \min\{C(S - \{j\}, i) + d_{ij} : i \in S, i \neq j\}$$

return $\min_j C(\{1, \dots, n\}, j) + d_{j1}$

$2^n \cdot n$ subproblems $\rightarrow$ Each Linear

Memory: $n2^n$

$$O(n^2 2^n)$$

Can BU have cheaper DP table?